

How to Learn the Optimal Clique Decompositions in Solving Semidefinite Relaxations for OPF

Charly Alizadeh*, Pegah Alizadeh[§], Miguel F. Anjos[‡], Lucas Létocart[§] and Emiliano Traversi[§]

**Léonard de Vinci Pôle Universitaire, Research Center, 92 916 Paris La Défense, France*

[‡]*University of Edinburgh, Edinburgh, United Kingdom*

charly-kyan.alizadeh_asle_saisan@edu.devinci.fr; miguel.f.anjos@ed.ac.uk

[§]*LIPN, UMR CNRS 7030, Université Sorbonne Paris Nord, Villetaneuse, France*

{alizadeh; letocart; traversi}@lipn.univ-paris13.fr

Abstract—The Optimal Power Flow (OPF) problem is a central optimization problem in power systems. Its global resolution is a challenge since it is highly nonconvex and NP-hard. Semidefinite Programming (SDP) is a powerful tool to progress towards global optimality as semidefinite relaxations provide tight lower bounds for the OPF problem. However, solving semidefinite relaxations for large power networks is very costly, because it is required to exploit its sparsity for achieving this aim. One efficient way to exploit sparsity for the OPF problem is to use clique decomposition techniques along with state-of-the-art interior point algorithms. Yet many clique decompositions can be computed for the same sparse SDP problem, their performance can significantly varies in practice. In this context, it is crucial to identify a *good* decomposition, where by *good* we mean a decomposition that allows to solve the SDP relaxation of the OPF problem in a small amount of time. At the moment, it is not possible in the literature to find a systematic analysis that allows to characterize in detail the properties of a good decomposition, the works proposed so far relies on the basic assumption that there is a trade-off between the size and the number of the cliques: a decomposition with only one large clique is problematic because of memory issues but a decomposition with many tiny cliques is not advisable either as it implies lots of linking constraints, which slows down the resolution. In this work, we propose to use machine learning techniques to understand what are the characteristics of a good clique decomposition. More precisely, we propose to identify the relevant features to describe a good clique decomposition, using both classification and regression approaches. The results show that the decomposition identified with the proposed techniques are comparable with the state of the art.

Index Terms—Classification for Optimisation, Regression for Optimisation, Clique decomposition, Optimal Power Flow, Semidefinite programming.

I. INTRODUCTION

The Optimal Power Flow (OPF) problem is a famous power systems problem that has been studied for sixty years [4]. It consists in determining a generation dispatch that meets the electrical demand and satisfies transmission network constraints while minimizing generation costs. Its global resolution is a challenge as it is highly nonconvex and NP-hard [3], [12]. Yet many promising exact methods have been proposed. Most of them make extensively use of semidefinite relaxations, as they provide tight lower bounds for OPF

problems. The technique used to solve semidefinite relaxations is Semidefinite Programming (SDP) [7], [11].

There are several options to solve a large SDP problem. One widely-used approach that works well in practice for OPF problems is to reformulate the semidefinite relaxation by using the so-called *clique decomposition* techniques [6], [16], [22]. One advantage of clique decomposition techniques is that they are pre-treatment procedures, that make it possible to use state-of-the-art SDP solvers. The basic idea of clique decomposition techniques is to reformulate an SDP problem in a way that allows the SDP solvers to work with several small semidefinite matrices, rather than one single large semidefinite matrix. Clique decomposition techniques have been widely used to solve semidefinite relaxations of OPF problems [5], [10], [14]. Different approaches are possible, since there are several admissible clique decompositions for the same SDP problem. These different clique decompositions are not equivalent in terms of solution time. Among the available techniques in the literature for clique decomposition, the more promising are the ones based on clique merging algorithms [14], [20]. The only approach in the literature that combines learning techniques and optimization for clique decomposition consists in learning chordal extensions [13], in order to simulate a minimum degree policy for obtaining a valid decomposition.

Paper contribution. The aim of this work is to obtain a predictor that allows to a-priori identify clique decomposition for which the resolution time with state-of-the-art SDP solver is the fastest. We explain in detail all the necessary steps (feature selection, dataset generation, labeling and evaluation technique) that allows to detect decompositions with a better performance than the decompositions obtained with the current state of the art.

The rest of this paper is organized as follows. Section II describes the OPF formulation. The semidefinite relaxation used for this problem is presented in Section III. Section IV details clique decomposition techniques. Our machine learning method is described in Section V. Computational results are shown in this section as well. Finally Section VI concludes the paper.

Identify applicable funding agency here. If none, delete this.

II. THE OPTIMAL POWER FLOW PROBLEM

The Optimal Power Flow (OPF) problem is an optimization problem based on a power transmission network that determines all the physical quantities such as the voltages, the currents and the powers in this network. Alternating Current (AC) is widely used for power transmission, which implies that all these physical quantities are complex. Therefore, the OPF problem is modeled with complex quantities and variables.

A power transmission network can be seen as an oriented graph (N, B) . The nodes in N , called buses, represent electrical nodes which can generate and/or consume power. A bus that can generate power is called a *generator bus*. The set of generator buses is noted $G \subset N$. The arcs in B , called branches, represent transmission lines and/or transformers.

The OPF problem consists of two types of complex variables as following:

- A complex voltage $v_n \in \mathbb{C}$ at each bus $n \in N$;
- A complex generation power variable $S_g \in \mathbb{C}$ at each generator bus $g \in G$.

Note that the real part of a complex power is called the active power and the imaginary part the reactive power.

An OPF problem should respect several constraints as well. **Power flow equations constraints.** The first constraints of the OPF problem are the power flow equations. They ensure, for all buses in N , that the power coming in a bus is equal to the power coming out. More precisely, at a bus $n \in N$, the generated power S_n (which is 0 if $n \notin G$) is equal to the demand S_n^l (the complex load), plus the power of the entering branches, minus the power of the exiting branches.

The power at the origin of a branch $l = (o, d)$ depends on the voltage variables v_o and v_d and is equal to:

$$S_l^{orig} = \frac{\bar{y}_l - \mathbf{j}b_l}{\tau_l^2} |v_o|^2 - \bar{y}_l \frac{e^{\mathbf{j}\theta_l}}{\tau_l} v_o \bar{v}_d$$

with the following constant parameters:

- the admittance y_l ,
- the charging susceptance b_l ,
- the transformer ratio τ_l ,
- the transformer phase shift angle θ_l .

The power at the destination of a branch $l = (o, d)$ depends on the same variables and parameters but has a different expression:

$$S_l^{dest} = -\bar{y}_l \frac{e^{-\mathbf{j}\theta_l}}{\tau_l} \bar{v}_o v_d + (\bar{y}_l - \mathbf{j}b_l) |v_d|^2.$$

Note that the powers at the origin and at the destination of the same line are not equal because of the resistive losses. Also note that the power flow equations are mainly responsible for the nonconvexity of the problem.

Generation power constraints. The second constraints for the OPF problem are constraints on the generation power S_g at each generator bus $g \in G$. There are bounds P_g^{min}, P_g^{max} on the real part and bounds Q_g^{min}, Q_g^{max} on the imaginary part of the generation variables. They correspond to physical limits of the generation means.

Voltage magnitude constraints. The third constraints for the OPF problem are constraints on the voltage magnitude. At each bus $n \in N$, the voltage magnitude $|v_n|$ must be between v_n^{min} and v_n^{max} which are the normal operating ranges.

Thermal limits constraints. The last constraints are thermal limits on branches. They are expressed in terms of current magnitude. The current magnitude at the origin and at the destination of a branch $l = (o_l, d_l)$ must be less or equal to i_l^{max} . The current at the origin of a branch $l = (o_l, d_l)$ depends on the voltage variables v_{o_l} and v_{d_l} and on branch parameters that have been described above. It is equal to :

$$i_l^{orig} = \frac{\bar{y}_l + \mathbf{j}b_l}{\tau_l^2} v_{o_l} - \bar{y}_l \frac{e^{-\mathbf{j}\theta_l}}{\tau_l} v_{d_l}.$$

Similarly, the current at the destination of a branch $l = (o_l, d_l)$ depends on the same variables and parameters, which is equal to :

$$i_l^{dest} = -\bar{y}_l \frac{e^{\mathbf{j}\theta_l}}{\tau_l} v_{o_l} + (\bar{y}_l + \mathbf{j}b_l) v_{d_l}.$$

Note that these thermal limits can be expressed with the power on the lines instead of the current but using a current modelling is more realistic.

The objective function represents the generation costs. It is a linear function of the active power generation $Re(S_g)$. Generation costs are usually expressed with quadratic functions but the linear costs more reflect the practice at RTE (the French transmission system operator), for which the objective is rather to minimize the deviation from a given generation dispatch.

To summarize, the OPF problem is the following problem:

$$\begin{aligned} \min_{v, S} \quad & \sum_{g \in G} c_g Re(S_g) + k_g \\ \text{s.t.} \quad & S_n = S_n^l + \sum_{l=(n,d)} \left(\frac{\bar{y}_l - \mathbf{j}b_l}{\tau_l^2} |v_n|^2 - \bar{y}_l \frac{e^{\mathbf{j}\theta_l}}{\tau_l} v_n \bar{v}_d \right) \\ & - \sum_{l=(o,n)} \left(-\bar{y}_l \frac{e^{-\mathbf{j}\theta_l}}{\tau_l} \bar{v}_o v_n + (\bar{y}_l - \mathbf{j}b_l) |v_n|^2 \right) \quad \forall n \in N \\ & P_n^{min} \leq Re(S_n) \leq P_n^{max} \quad \forall n \in G \\ & Q_n^{min} \leq Im(S_n) \leq Q_n^{max} \quad \forall n \in G \\ & S_n = 0 \quad \forall n \notin G \\ & (v_n^{min})^2 \leq |v_n|^2 \leq (v_n^{max})^2 \quad \forall n \in N \\ & \left| \frac{\bar{y}_l + \mathbf{j}b_l}{\tau_l^2} v_{o_l} - \bar{y}_l \frac{e^{-\mathbf{j}\theta_l}}{\tau_l} v_{d_l} \right|^2 \leq (i_l^{max})^2 \quad \forall l \in B \\ & \left| -\bar{y}_l \frac{e^{\mathbf{j}\theta_l}}{\tau_l} v_{o_l} + (\bar{y}_l + \mathbf{j}b_l) v_{d_l} \right|^2 \leq (i_l^{max})^2 \quad \forall l \in B \\ & v_n \in \mathbb{C}, S_n \in \mathbb{C} \quad \forall n \in N. \end{aligned} \quad (1)$$

In this formulation, the power generation variables can be eliminated. The costs being linear, the resulting problem is a nonconvex Quadratically Constrained Quadratic Problem (QCQP) with complex variables.

III. THE RANK RELAXATION

The rank relaxation is the most common SDP relaxation for a nonconvex QCQP. A generic nonconvex QCQP with complex variables is described below: the variable of the

problem is the complex vector v and the objective and the constraints are quadratic in v . The rank relaxation is constructed by introducing the Hermitian matrix $W = vv^H$, a complex square matrix that is equal to its own conjugate transpose. The objective and the constraints are then linear in W . The constraint linking W and v is equivalent to $W \succeq 0$ (W is positive semidefinite) and $\text{rank}(W) = 1$. Now, the nonconvexity of the problem is fully contained in the rank constraint. It suffices to suppress this rank constraint to get a convex relaxation. The construction of the rank relaxation is summarized in (2).

$$\begin{cases} \min & v^H Q_0 v \\ \text{s.t.} & v^H Q_p v \leq a_p \quad \forall p \\ & v \in \mathbb{C}^n \end{cases} \Rightarrow \begin{cases} \min & Q_0 \cdot W \\ \text{s.t.} & Q_p \cdot W \leq a_p \quad \forall p \\ & W \succeq 0 \end{cases} \quad (2)$$

The obtained SDP relaxation is expressed with complex variables but it can be converted to an SDP problem with real variables thanks to the rectangular representation of the complex numbers and by doubling the size of the positive semidefinite variable. Besides, this SDP relaxation can be reformulated using clique decomposition to speed up the resolution.

IV. CLIQUE DECOMPOSITION

Clique decomposition techniques are based on matrix completion [6], [16], [22], which is the task of filling in the missing entries of a partially observed matrix. The key idea is to replace the Positive Semidefinite (PSD) constraint on a large matrix by several PSD constraints on smaller submatrices. This is possible under some conditions (chordality of a graph) and at the price of adding linking constraints between the submatrices.

In order to compute a clique decomposition for a sparse SDP problem, four steps are necessary. First, the aggregate sparsity pattern A of the problem has to be constructed: this is the matrix that summarizes all the nonzeros entries in the data matrices. This aggregate sparsity pattern can be seen as a graph G^A if we consider it as an adjacency matrix. The second step consists in computing a chordal extension H of the graph G^A .

Definition 1: A chordal graph is a graph with no induced cycles of size 4 or more.

Having a chordal graph is mandatory to get an equivalent SDP reformulation. There are several ways to compute a chordal extension of a graph. In many applications, it is interesting to compute a chordal extension with as few additional edges as possible. Although finding the minimal chordal extension of a graph is NP-complete [23] but there exists a lot of efficient heuristics [8] for this aim. The commonly used heuristic for the OPF problem is a Cholesky factorization [18], [19] preceded by an Approximate Minimum Degree (AMD) ordering [1]. The third step consists in listing all the maximal cliques in chordal extension H .

Definition 2: A clique is a subset of vertices that are all connected together. A maximal clique is a clique that is not contained in another clique.

Remark 1: Determining the list of maximal cliques $L = \{C_1, \dots, C_r\}$ can be done in linear time since H is chordal [21].

The maximal cliques represent the submatrices that have to be PSD in the reformulation. The maximal cliques can overlap, that is why the last step focuses on linking constraints between the submatrices. One way to express them efficiently is to use a clique tree. To compute this *clique tree*, we need to define the clique graph W . The nodes of this graph are the maximal cliques in L and there is a weighted edge between two nodes if the two corresponding cliques share some nodes. The weight of an edge (C_i, C_j) is the number of nodes that are shared by the maximal cliques C_i and C_j . A clique tree is a maximum-weight spanning tree of W . It can be computed using Prim's algorithm [17].

If the resulting clique decomposition contains lots of tiny cliques, which happens for very sparse problems as the semidefinite relaxations of OPF problems, post-treatment procedures such as clique merging can be helpful. Clique merging consists in adding additional edges in the chordal graph in order to bring together two cliques that share a certain number of nodes. Merging cliques therefore allows to reduce the number of linking constraints. It can be done under certain conditions, otherwise the chordal property can be lost. As far as we know, all clique merging algorithms merge cliques that are adjacent in the clique tree to ensure the chordality of the resulting graph.

For the rank relaxation of the OPF problem, it is interesting to work on the complex SDP problem and then converting the clique decomposition with complex variables into a clique decomposition with real variables by doubling the size of the cliques. In this case, G^A corresponds to the network graph.

V. LEARNING GOOD CLIQUE DECOMPOSITIONS

In this section we explain how to design a system that receives a clique decomposition for a SDP problem and predicts its quality in terms of solution time. In order to design such a system, the following questions need to be addressed:

- Q1 How should an OPF instance be represented? How should a clique decomposition be represented? In other words, what are the representative features that should be considered?
- Q2 What is the most effective way to label a decomposition? More precisely: what kind of normalization should be used in a regression method and what kind of labelling should be used in a classification method?
- Q3 In order to train an intelligent system, how to generate a balanced training dataset?
- Q4 What is the most effective architecture for a classifier and for a regressor?
- Q5 How should the classifier/regressor be used in practice and how should we evaluate if such a use is effective?

In the rest of the paper we give an answer to each of the five mentioned research questions.

A. Features selection

One peculiar aspect of this application is that each point in our data set consists of a pair instance-decomposition, where an instance represents a specific SDP relaxation of an OPF problem, while with the decomposition we mean one of its possible clique decompositions. This implies that all the points corresponding to the same instance share the same value for a subset of their features. The features can be grouped into two families:

- *Network-based features.* These features depend only on the aggregate sparsity pattern of the network graph G^A .

- The number of vertices: $|V_{G^A}|$.
- The number of edges: $|E_{G^A}|$.
- The maximum degree¹: $\max_{i \in V_{G^A}} d(i)$.
- The minimum degree: $\min_{i \in V_{G^A}} d(i)$.
- The mean degree: $\frac{\sum_{i \in V_{G^A}} d(i)}{|V_{G^A}|}$.
- The global clustering coefficient:

$$\frac{\text{number of triangles in } G^A}{\sum_{i \in V_{G^A}} \binom{d(i)}{2}}.$$

- The density of G^A .
- The diameter of G^A .
- The radius of G^A .

- *Decomposition features.* These features are used to extract the main properties of one clique decomposition, in relation to the instance. We can further split an instance's features as follows:

- *Extension-based features.* Similar to the Network-based features, these features depend only on the aggregate sparsity pattern of the chordal extension H . They are the same as the Network-based instance features with H replaced by G^A .
- *Clique features.* These additional features are used to represent the characteristics of the cliques of the chordal extension:
 - * The number of cliques.
 - * The maximal size of a clique.
 - * The minimum size of a clique.
 - * The mean size of the cliques.
 - * The variance of the size of the cliques.

B. How to label an instance-decomposition pair

The solution time for one instance depends first of all on the solver used to solve it. Several software packages are available for solving SDP problems and the performance of a decomposition can vary substantially depending on the choice of solver. In our tests, we used the solver MOSEK [15], a commercial SDP solver that has been successfully used for solving SDP relaxations of OPF problems. We believe that,

¹The degree $d(i)$ of a node i represents the number of edges insisting on the node i

even if the results would possibly be different with a different solver, the procedure proposed is sufficiently generic to be applicable with other solvers.

For the set of OPF instances considered, the solution time without applying any clique decomposition can vary significantly (from minutes to hours). Furthermore, if a suitable choice of decomposition is used, the solution time can decrease by up to a factor of 10, while using a poor choice of decomposition the solution can be several orders of magnitude slower (or even become impossible to solve). It is therefore not advisable to use the solution time as a label for instance-decomposition pairs.

As mentioned in Section IV, the commonly used heuristic for relaxations of OPF is the combination of an Approximate Minimum Degree (AMD) ordering, followed by a Cholesky factorization.

Definition 3: Let $t_{Cholesky}$ be the solution time of the commonly used heuristic and $t_{decomposition}$ be the solution time of a given decomposition. The solution time of a decomposition can be normalized as follows:

$$T = (t_{decomposition} - t_{Cholesky})/t_{Cholesky}.$$

Based on this measure, we label how much faster (or slower) a given decomposition is by comparing its solution time with the time taken by the most commonly used technique. A decomposition with a label T less than 0 is considered a “good” decomposition.

C. Data Generation

In principle, we should generate a representative collection of decompositions for each SDP instance. A naive way to do this is to generate decompositions randomly. The main obstacles that make the generation of a diversified set of decompositions difficult are:

- The space of all the valid decompositions is exponentially large and therefore it is not possible to enumerate all of them. Figure 1 reports the correlation between the number of clique decompositions (enumerated by brute force) and the density² for a graph with 7 nodes. In our testset, the average density is less than 1%, this shows that it is impossible in practice to enumerate all the decompositions.
- The procedure needed to generate a valid clique decomposition is not trivial. This is because the set of valid chordal extensions represents only a small subset of all the valid extensions of a graph. For this reason it is not clear how to properly randomly generate one decomposition.

Moreover, even if we are able to obtain a set of feasible randomly generated decompositions, it is likely that the majority of them are bad, i.e., they are difficult to solve. As mentioned in Section V-B, the solution time of a bad decomposition can be up to several days. Therefore, it is important to define a

²By density of a graph we indicate the total number of edges divided by the total number of potential edges, i.e., $\frac{|E|}{|V| \cdot (|V|-1)/2}$

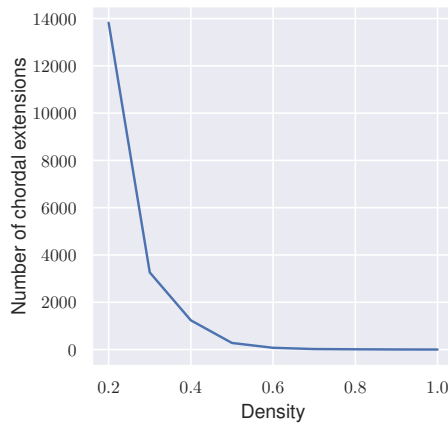


Fig. 1: Evolution of the number of chordal extensions of a graph with 7 nodes in function of its density.

procedure that systematically generates decompositions that are diversified *and* that are unlikely to have $T > 2$.

To generate a set containing a variety of decompositions (good or bad), we propose a procedure that mixes the majority of the clique decomposition algorithms that are available in the literature. Figure 2 gives an overview of the procedure used to generate the set of decompositions of a given instance.

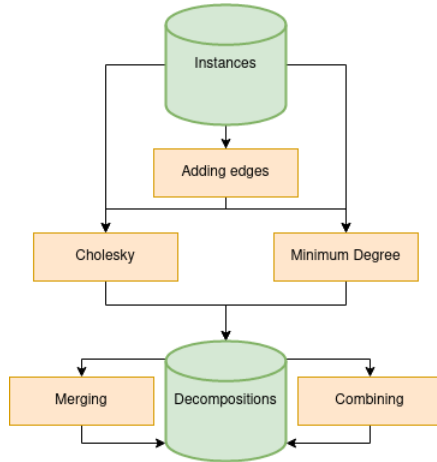


Fig. 2: Flowchart of generating decompositions for the training process

We begin by computing decompositions obtained with well-established techniques from the literature. In preliminary tests, we tested the following three chordal extension algorithms:

- Minimum Degree (MD) heuristic [2]. The first effective decomposition heuristic.
- Cholesky factorization preceded by an Approximate Minimum Degree (AMD) ordering. This is the most commonly used technique to obtain a chordal extension.
- The LEX-M [9] and the MCS-M algorithms [2].

Generally speaking, all three algorithms heuristically minimize the number of additional edges. We found that the decompositions obtained by applying LEX-M and MCS-M

were constantly outperformed by those from the other two algorithms. We therefore decided to only use the first two algorithms.

To increase diversity, we also generate decompositions after applying Minimum degree and Cholesky to a perturbed version of the instances created by randomly adding additional edges. We performed two types of edge perturbations:

- *Light perturbation.* For each instance, we randomly choose 20 edges that do not belong to the original graph to add among the set of edges for which the distance in the original graph between their two nodes is equal to 2. We repeat this random procedure three times, to get 3 different graphs per instance.
- *Heavy perturbation.* For each instance, we randomly add 50 edges to the graphs. This procedure is repeated twice per instance.

The light perturbation is aimed at obtaining decompositions that are more likely to be good, while the heavy perturbation is more likely to provide representatives of bad decompositions. Then, we apply Minimum degree and Cholesky to the five perturbed instances to compute a chordal extension. In summary, in the first step of the procedure, we obtain up to $5+2=7$ clique decompositions per instance (we can obtain fewer if some of them are identical).

Next we use different clique merging and clique combination algorithms in order to generate more good clique decompositions. We use two state-of-the-art clique merging heuristics present in the literature: the clique merging heuristic proposed in [14] and the one proposed in [20]. We propose different stopping criteria: from 90% of the number of cliques to 10% of the number of cliques with a 10% step, which means generating 9 clique decompositions with each clique merging algorithm. In total, we generate at most $8 \times 9 \times 2 = 144$ clique decompositions per instance.

Algorithm 1 is used to combine two decompositions. The basic idea of the algorithm is to take either the union or the intersection of their edges. For the latter we need to check if the obtained decomposition is valid, and if this is the case, we apply Minimum degree and Cholesky to obtain valid clique decompositions.

The mentioned two-step procedure generates on average 271 clique decompositions for each instance.

Algorithm 1 Chordal graph combination

Input: $G_1 = (V, E_1)$ and $G_2 = (V, E_2)$ chordal graphs

Output: $G_3 = (V, E_3)$ the combination of G_1 and G_2

- 1: $G_3 = (V, E_1 \cap E_2)$
 - 2: **if** G_3 is not chordal **then**
 - 3: Make G_3 chordal
 - 4: **end if**
 - 5: **return** G_3
-

D. Experimental results for Learning Good Decompositions

After explaining how to extract the features, generate the training set, and how to label those samples, we are now able

to train a prediction model on the prepared training set and predict weather a decomposition is good or not for any given SDP instance. Such a prediction can be performed both as classification and as regression. In both cases, the instances used in our experiments are taken from the MATPOWER library [24], a well established collection of OPT instances. The library consists of a total of 49 instances from which we only took the instances with a number of nodes between 1000 and 15000 which corresponds to 22 instances. To obtain a more accurate estimation of the classifier and regression models, we perform cross validation as following: the total set of instances is split into 7 folds, where each instance has 271 decompositions in average. The testing set of each fold contain decompositions of instances not present in the training set and each instances is included in exactly one test set of a fold.

1) *Classification*: For classification purposes, we train a binary classification, where the decompositions with a value of T lower than -0.35 are labeled as *good*, while the others are labeled as *bad*.

We test three candidates for the best classifier: Random Forest (RF), Support Vector Machine (SVM) and adaboost (ADAB). Table I shows the accuracy, F1 scores, precision and recalls for the three tested classifiers on the testing set. Between the three model we give the priority to the one with the higher precision, this is because we are interested in having a model able to not be wrong when a decomposition is labelled as “good”. For this reason we decide to used adabost as classifier. The adaboost classifier uses a decision tree as its base estimator with the default parameters of the scikit-learn library, 500 estimators, a learning rate of 0.1 and the SAMME.R algorithm.

TABLE I: Scores of the classification models (testing set)

Model	Accuracy	F1	Precision	Recall
ADAB	0.68	0.53	0.69	0.45
RF	0.65	0.38	0.66	0.28
SVM	0.70	0.56	0.65	0.52

From the point of view of an optimizer, the indicators provided in Table I do not give an indication about how the classifier could perform in practice. For this reason, a second test is needed, to simulate how the classier could be used to guide the choice of a decomposition for an unknown instance. Table II shows an attempt to measuring the “guiding-potential” of the trained classifier. In other words, how effectively it can be used to guide in the selection of a good decomposition. The column *Name* lists the set of instances, sorted by the id of their fold. The column *Count* shows the number of decompositions labelled as “good” by the classifier (over the set of all possible decompostions generated). The column *AvgPred* and *MinPred* show the average and minimum value of the solving time over the instances labelled as “good”. The column *Chlsky* reports the solving time obtained by the decomposition provided by the algorithm *Cholesky*, while the column *Merge* reports the solving time obtained by the decomposition provided by the state-of-the-art algorithm for clique decomposition described

in [20]. The goal of this test is to see how one decomposition chosen randomly among all the instance labelled as “good” would perform in comparison to the most used and the most effective algorithms for clique decomposition. The values reported in column *AvgPred* are systematically better than the ones provided by the column *Chlsky*, this implies that the classification is able to provide solutions better than a a reasonably good algorithms. On the other hand, column *Merge* shows that the average results are not comparable with the ones provided by the best algorithm available. A possible explanation for this behaviour is that a classifier lacks the required granularity to identify the best decomposition, this intuition is supported by the results provided by column *MinPred*: among the instances labelled as “good” it is almost always possible to find a decomposition able to improve the state of the art, the problem is that a classifier is not able to identify it, this is due to the fact that with a classification task we cannot compare two decompositions with the same label if we don’t have their respective solution time. Those results motivated us to train a regression model in order to be able to compare two decompositions with the same label.

TABLE II: Results of the classification experiment (testing set)

Name	Count	AvgPred	MinPred	Chlsky	Merge
3120sp(0)	215	420	205	854	441
6515rte(0)	91	714	362	1077	476
ACTIVSg2000(0)	0	NaN	NaN	1969	510
2383wp(1)	94	219	123	376	185
2868rte(1)	52	101	50	139	90
6495rte(1)	180	694	285	1211	521
2737sop(2)	34	314	130	462	301
2746wop(2)	49	310	195	614	315
6468rte(2)	161	663	343	1493	491
1354pegase(3)	3	140	131	62	31
3012wp(3)	174	387	170	629	405
9241pegase(3)	195	1247	283	2329	625
1951rte(4)	61	66	22	68	46
3375wp(4)	273	383	170	642	335
6470rte(4)	125	753	259	1085	398
2736sp(5)	98	308	169	425	321
2746wp(5)	97	355	159	480	300
2848rte(5)	82	110	44	129	85
1888rte(6)	11	48	30	47	24
2869pegase(6)	36	224	79	136	97
ACTIVSg10k(6)	74	2883	1448	5168	2426
13659pegase(6)	200	964	326	2219	461

2) *Regression task*: For the regression task we tested four possible models: Adaboost (ADAB), Random Forest (RF), Support Vector Regression (SVR) and a Feed Forward Neural Network (FFNN). The neural network consist of a single hidden layer with 64 neurons and a dropout of 0.5 after each layer. In preliminary tests, we tested for each model two different options of labelling: by normalizing the solving time (and using therefore T) or by using the original solving time. The results obtained with the normalized targeting where almost systematically less accurate. In Figure 3 (respectively Figure 4) we show how much the predicted values deviate from the real values when the original (respectively normalize) solution time is used by training the adaboost regression method (similar figures can be obtained for the other methods). The two figures clearly show that the use of the normalized



Fig. 3: Visualization of the predictions on the testing set (with normalized target). The x-axis represents the real time labels and y-axis the predicted time.

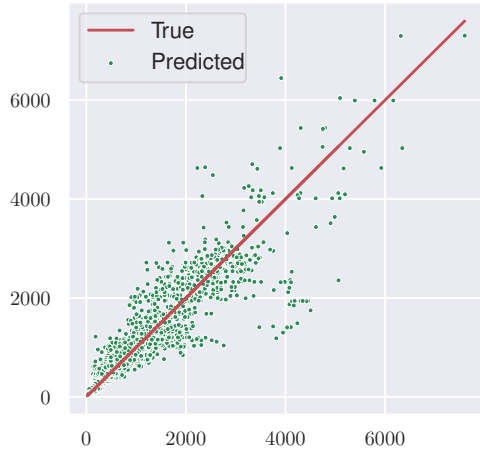


Fig. 4: Visualization of the testing set predictions. The x-axis represents the real time labels and y-axis the predicted one.

solving time leads to less accurate predictions, for this reason, we do not use any normalization in any of the regressions. To evaluate the four models, we used the Pearson correlation and the R^2 metrics to evaluate our models. The Pearson correlation was applied on the true targets ordered in function of their predicted values and $[0, \dots, N]$ where N is the number of observations. This correlation helped us determine how well ordered our predictions were. The motivation behind the use of this metric is that even if fitting the real targets would be better, if our model can tell which decomposition is better, it is a more effective information in order to choose which decomposition should be used. Our models' performances are depicted in Table III with two measures Pearson and R^2 . The

only two models with a good R^2 values are adaboost and random forest. With preliminary tests we experience that both provide similar results. Therefore for coherence we decide to use adaboost also as regression model. The adaboost regressor uses a decision tree as its base estimator where the split at each node is done randomly, 100 estimators, a learning rate of 0.1 and a square loss.

TABLE III: Measures for evaluating the regression models on the test set.

Model	PearsonCorr	R^2
ADAB	0.57	0.19
RF	0.57	0.19
SVR	0.61	-0.06
FFNN	0.65	-0.92

Also for the regression case, we performed a second experiment to evaluate the “guiding-power” of the selected regression model. The results of this experiment can be found on Table IV. Columns *Name*, *Chlsky* and *Merge* have the same meaning of Table II. Column *Avg_k* reports the average computing times of the first k decompositions with the best score provided by the regression model. In other words, *Avg₁* provides the computing time obtained after applying the decomposition with the best score from the regression model, *Avg₅* and *Avg₁₀* the average computing power obtained with the first five and ten ranked decompositions. The results show that in 16 out of 22 instances the regression method is able to provide a decomposition that is better than the decomposition obtained with a state-of-the-art algorithm. The good results provided by columns *Avg₅* and *Avg₁₀* show that the regression method is also robust, i.e., not only the decomposition labelled with the best score provides good results, but also the first five and ten identified decomposition are effective.

Figure 5 shows how the decompositions provided by our method is faster (positive values) or slower (negative values) with respect to the decomposition obtained by the algorithm proposed in [20]. The figure shows that in all the folds our method provides the fastest decomposition in at least two instances.

VI. CONCLUSION AND FUTURE WORKS

In this work, we showed how Machine Learning can help in improving the solution time of Semidefinite Programming problems by improving the choice of an effective clique decompositions techniques. More precisely, we designed a regression model that receives the SDP problem and a related clique decomposition, and it predicts if the proposed clique decomposition will solve the problem in a reasonable time or if the problem is almost unsolvable in a reasonable amount of time. From an optimisation point of view, having this information is essential because it helps in finding a good solution for the SDP problem, as we showed in the experimental results of this work. This work represents a first promising step towards integrating machine learning techniques into algorithms for the resolution of semidefinite problems. In the future, we are interested in investigating how to better represent the features

TABLE IV: Results of the regression experiment on the testing folds in cross validation implied on the instances.

Name(fold)	Avg ₁	Avg ₅	Avg ₁₀	Chlsky	Merge
3120sp(0)	312	355	345	854	441
6515rte(0)	362	612	612	1077	476
ACTIVSg2000(0)	343	343	363	1969	510
2383wp(1)	201	201	201	376	185
2868rte(1)	63	73	75	139	90
6495rte(1)	504	459	489	1211	521
2737sop(2)	163	237	239	462	301
2746wop(2)	281	312	311	614	315
6468rte(2)	620	524	494	1493	491
1354pegase(3)	35	35	35	62	31
3012wp(3)	248	291	300	629	405
9241pegase(3)	485	591	679	2329	625
1951rte(4)	29	38	40	68	46
3375wp(4)	322	266	249	642	335
6470rte(4)	588	494	531	1085	398
2736sp(5)	231	252	252	425	321
2746wp(5)	264	304	289	480	300
2848rte(5)	72	72	81	129	85
1888rte(6)	34	34	37	47	24
2869pegase(6)	85	87	104	136	97
ACTIVSg10k(6)	1850	2383	2332	5168	2426
13659pegase(6)	552	648	694	2219	461

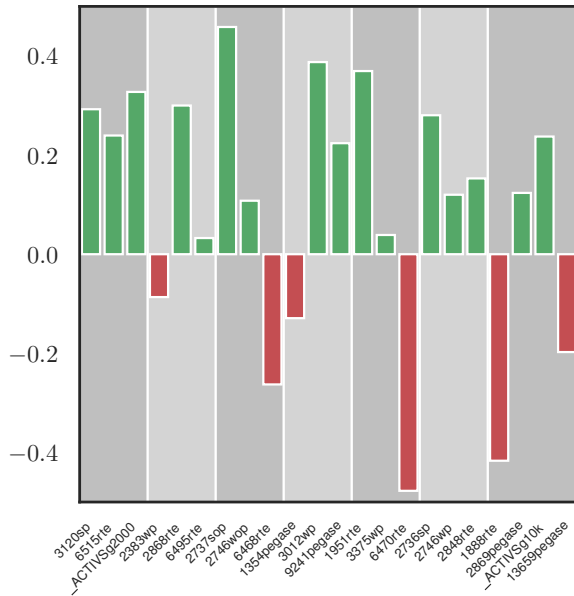


Fig. 5: Histogram of the ratio of improvement of Avg_1 over $merge$. The different folds are shown on the x-axis, while the y-axis this ratio of $\frac{merge - avg_1}{merge}$.

in our model and in testing our model on a significantly larger set of instances.

VII. ACKNOWLEDGEMENT

This work has been partially funded by the “Fondation Mathématique Jacques Hadamard” with the PGMO – IROE grant number P-2021-0019.

REFERENCES

[1] P. R. Amestoy, T. A. Davis, and I. S. Duff. An approximate minimum degree ordering algorithm. *SIAM Journal on Matrix Analysis and Applications*, 17(4):886–905, 1996.

[2] B. Anne, B. J. R. S., H. Pinar, and P. B. W. Maximum cardinality search for computing minimal triangulations of graphs. *Algorithmica*, 39(4):287–298, 2004.

[3] D. Bienstock and A. Verma. Strong np-hardness of ac power flows feasibility. *Operations Research Letters*, 2019.

[4] J. Carpentier. Contribution to the economic dispatch problem. *Bulletin de la Societe Francaise des Electriciens*, 3(8):431–447, 1962.

[5] A. Eltvéd, J. Dahl, and M. S. Andersen. On the robustness and scalability of semidefinite relaxation for optimal power flow problems. *Optimization and Engineering*, 21(2):375–392, 2020.

[6] M. Fukuda, M. Kojima, K. Murota, and K. Nakata. Exploiting sparsity in semidefinite programming via matrix completion i: General framework. *SIAM Journal on Optimization*, 11(3):647–674, 2001.

[7] A. Gopalakrishnan, A. U. Raghunathan, D. Nikovski, and L. T. Biegler. Global optimization of optimal power flow using a branch & bound algorithm. In *2012 50th Annual Allerton Conference on Communication, Control, and Computing (Allerton)*, pages 609–616. IEEE, 2012.

[8] P. Heggernes. Minimal triangulations of graphs: A survey. *Discrete Mathematics*, 306(3):297–317, 2006.

[9] R. D. J. and T. R. Endre. Algorithmic aspects of vertex elimination on directed graphs. *SIAM Journal on Applied Mathematics*, 34(1):176–197, 1978.

[10] R. A. Jabr. Exploiting sparsity in sdp relaxations of the opf problem. *IEEE Transactions on Power Systems*, 27(2):1138–1139, 2012.

[11] C. Jozs, J. Maeght, P. Panciatici, and J. C. Gilbert. Application of the moment-sos approach to global optimization of the opf problem. *IEEE Transactions on Power Systems*, 30(1):463–470, 2015.

[12] K. Lehmann, A. Grastien, and P. Van Hentenryck. Ac-feasibility on tree networks is np-hard. *IEEE Transactions on Power Systems*, 31(1):798–801, Jan 2016.

[13] D. Liu, A. Lodi, and M. Tanneau. Learning chordal extensions. *Journal of Global Optimization*, pages 1–20, 2021.

[14] D. K. Molzahn, J. T. Holzer, B. C. Lesieutre, and C. L. DeMarco. Implementation of a large-scale optimal power flow solver based on semidefinite programming. *IEEE Transactions on Power Systems*, 28(4):3987–3998, 2013.

[15] Mosek. The mosek optimization software. *Online at http://www.mosek.com*, 2019.

[16] K. Nakata, K. Fujisawa, M. Fukuda, M. Kojima, and K. Murota. Exploiting sparsity in semidefinite programming via matrix completion ii: Implementation and numerical results. *Mathematical Programming*, 95(2):303–327, 2003.

[17] R. C. Prim. Shortest connection networks and some generalizations. *The Bell System Technical Journal*, 36(6):1389–1401, 1957.

[18] D. J. Rose. Triangulated graphs and the elimination process. *Journal of Mathematical Analysis and Applications*, 32(3):597–609, 1970.

[19] D. J. Rose. A graph-theoretic study of the numerical solution of sparse positive definite systems of linear equations. In *Graph theory and computing*, pages 183–217. Elsevier, 1972.

[20] J. Sliwak, E. D. Andersen, M. F. Anjos, L. Létocart, and E. Traversi. A clique merging algorithm to solve semidefinite relaxations of optimal power flow problems. *IEEE Transactions on Power Systems*, 36(2):1641–1644, 2021.

[21] R. E. Tarjan and M. Yannakakis. Simple linear-time algorithms to test chordality of graphs, test acyclicity of hypergraphs, and selectively reduce acyclic hypergraphs. *SIAM Journal on computing*, 13(3):566–579, 1984.

[22] L. Vandenbergh, M. S. Andersen, et al. Chordal graphs and semidefinite optimization. *Foundations and Trends in Optimization*, 1(4):241–433, 2015.

[23] M. Yannakakis. Computing the minimum fill-in is np-complete. *SIAM Journal on Algebraic Discrete Methods*, 2(1):77–79, 1981.

[24] R. D. Zimmerman, C. E. Murillo-Sánchez, and R. J. Thomas. Matpower: Steady-state operations, planning, and analysis tools for power systems research and education. *IEEE Transactions on Power Systems*, 26(1):12–19, 2011.